

LAB-04

Name: Syed Muhammad Abdullah

Roll no: AI – 22302

Exercises:

1. Create a class 'Employee' having two data members 'EmployeeName'(char*) and 'EmployeeId' (int). Keep both data members private. Create three initialized objects 'Employee1', 'Employee2' and 'Employee3' of type 'Employee' in such a way that the employee name for each employee can be changed when required but the employee Id for each employee must be initialized only once and should remain same always. Use member initializer list, accessors/getters and mutators/setters for appropriate data members. The result must be displayed by calling the accessors.

C++ CODE:

```
#include <iostream>
#include <cstring>
using namespace std;

class Employee{
private:
    const int EmployeeId;
    char* EmployeeName;

public:
    Employee(int Id, const char* Name): EmployeeId(Id) {
        EmployeeName = new char[strlen(Name)+1];
        strcpy(EmployeeName, Name);
    }
    void setName(const char* name){
        delete[] EmployeeName;
        EmployeeName = new char[strlen(name)+1];
        strcpy(EmployeeName, name);
    }
    int getEmpId(){
        return EmployeeId;
    }
    char* getEmpName(){
        return EmployeeName;
    }
};
```

```

int main(){
    Employee Employee1(300, "Ali");
    Employee Employee2(301, "Asad");
    Employee Employee3(302, "Abdullah");
    cout << "Information of Employee 1: ID: " << Employee1.getEmpId() << " Name: "
    << Employee1.getEmpName() << endl;
    cout << "Name of Employee 2 before modification: " << Employee2.getEmpName()
    << endl;
    Employee2.setName("Hamza");
    cout << "Name of Employee 2 after modification: " << Employee2.getEmpName()
    << endl;
    return 0;
}

```

OUTPUT:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\PMLS\Desktop\NED\8th Semester\OOP\Lab4> g++ ex1.cpp -o ex1
PS C:\Users\PMLS\Desktop\NED\8th Semester\OOP\Lab4> ./ex1
Information of Employee 1: ID: 300 Name: Ali
Name of Employee 2 before modification: Asad
Name of Employee 2 after modification: Hamza
PS C:\Users\PMLS\Desktop\NED\8th Semester\OOP\Lab4>

```

2. Create a class called DynamicArray. The class should contain a pointer to the array, and size of the array as data members. Create the parameterized constructor which take size of the array as input and initializes all the values with 0. Create the member function “push” which takes value as parameter and push it to the end of the array. Create the member function size() which returns the size of the array.

C++ CODE:

```

#include <iostream>
#include <string>
using namespace std;

class Dynamic_Array{
private:
    int* arr;
    int size;

public:
    Dynamic_Array(int Size): size(Size){

```

```

        arr = new int[size];
        for (int i=0; i<size; i++){
            arr[i] = 0;
        }
    }

    int getSize() const{
        return size;
    }

    void push(int value){
        int* newArr = new int[size+1];
        for (int j=0; j<size; j++){
            newArr[j] = arr[j];
        }
        newArr[size] = value;
        delete[] arr;
        arr = newArr;
        size++;
    }

    void displayArray(){
        cout << "Array: ";
        for (int k=0; k<size; k++){
            cout << arr[k] << " ";
        }
    }
};

int main(){
    Dynamic_Array dArray(4);
    cout << "Array size: " << dArray.getSize() << endl;
    cout << "Default Array: " << endl;
    dArray.displayArray();
    cout << "Pushing a value in the array..." << endl;
    dArray.push(67);
    cout << "New Array: " << endl;
    dArray.displayArray();
    return 0;
}

```

OUTPUT:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\PMLS\Desktop\NED\8th Semester\OOP\Lab4> g++ ex2.cpp -o ex2
PS C:\Users\PMLS\Desktop\NED\8th Semester\OOP\Lab4> ./ex2
Array size: 4
Default Array:
Array: 0 0 0 0 Pushing a value in the array...
New Array:
Array: 0 0 0 0 67
PS C:\Users\PMLS\Desktop\NED\8th Semester\OOP\Lab4> █
```

3. Write a program in which a class named Account has private member variables named account_no ,account_bal ,security_code. Use a public function to initialize the variables and print all data. Keep track of the number of objects using the keyword static.

C++ CODE:

```
#include <iostream>
#include <string>
using namespace std;

class Account{
    private:
        static int objectsCount;
        int account_no, security_code;
        double account_bal;

    public:
        static void displayObjectsCount(){
            cout << "The total accounts created: " << objectsCount << endl;
        }

        Account(int accountNum, double accountBalance, int securityCode){
            account_no = accountNum;
            account_bal = accountBalance;
            security_code = securityCode;
            objectsCount++;
        }

        void displayAccountInformation(){
            cout << "Account number: " << account_no << endl;
            cout << "Account balance: " << account_bal << endl;
            cout << "Security Code: " << security_code << endl;
        }
}
```

```

};

int Account::objectsCount = 0;

int main(){
    Account::displayObjectsCount();
    cout << "Creating Account 1: " << endl;
    Account account1(1, 10000, 380);
    account1.displayAccountInformation();
    cout << "Creating Account 2: " << endl;
    Account account2(2, 25000, 123);
    account1.displayAccountInformation();
    Account::displayObjectsCount();

    return 0;
}

```

OUTPUT:

```

PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\PMLS\Desktop\NED\8th Semester\OOP\Lab4> g++ ex3.cpp -o ex3
PS C:\Users\PMLS\Desktop\NED\8th Semester\OOP\Lab4> ./ex3
The total accounts created: 0
Creating Account 1:
Account number: 1
Account balance: 10000
Security Code: 380
Creating Account 2:
Account number: 1
Account balance: 10000
Security Code: 380
The total accounts created: 2
PS C:\Users\PMLS\Desktop\NED\8th Semester\OOP\Lab4>

```

- By considering a scenario of your own choice, write a program to demonstrate the concept of constant keyword.

Solution of the example scenario assumed:

The following program models a simple E-Commerce Order system.

Each order is assigned a unique **Order ID** at the time of creation, and this ID cannot be changed later. Therefore, the order ID is declared using the **const** keyword.

The class also stores the **total amount** of the order, which can increase when new items are added. A member function `addItem()` is provided to update the total amount.

A `display()` function is marked as **const** because it only shows the order details and does not modify any data members of the object.

This design demonstrates the use of:

- constant data members,
- initialization through the constructor, with constant argument,
- and constant member functions in object-oriented programming.

C++ CODE:

```
#include <iostream>
using namespace std;

class Order {
private:
    const int orderID;    // constant attribute
    double totalAmount;

public:
    // constructor
    Order(const int id, double amount) : orderID(id) {
        totalAmount = amount;
    }

    void addItem(double price) {
        totalAmount += price;
    }

    // constant function to make sure they don't modify object's data
    void display() const {
        cout << "Order ID: " << orderID << endl;
        cout << "Total Amount: " << totalAmount << endl;
    }
};

int main() {
    Order o1(4001, 5500);

    o1.addItem(300);
    o1.display();
}
```

```
    return 0;
}
```

OUTPUT:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\PMLS\Desktop\NED\8th Semester\OOP\Lab4> g++ ex4.cpp -o ex4
PS C:\Users\PMLS\Desktop\NED\8th Semester\OOP\Lab4> ./ex4
Order ID: 4001
Total Amount: 5800
PS C:\Users\PMLS\Desktop\NED\8th Semester\OOP\Lab4> █
```

5. “Hotel Mercato” requires a system module that will help the hotel to calculate the rent of the customers. You are required to develop one module of the system according to the following requirements:
- The hotel wants such a system that should have the feature to change the implementation independently of the interface. This will help when dealing with changing requirements.
 - The hotel charges each customer 1000.85/- per day. This amount is being decided by the hotel committee and cannot be changed fulfilling certain complex formalities.
 - The module then analyses the number of days. If the customer has stayed for more than a week in the hotel, he gets discount on the rent. Otherwise, he is being charged normally.
 - The discounted rent is being calculated after subtracting one day from the total number of days.
 - In the end, the module displays the following details: o Customer name o Days o Rent
Note that, the function used for displaying purpose must not have the ability to modify any data member.

C++ CODE:

```
#include <iostream>
#include <string>
using namespace std;

class Customer_Hotel_Mercato{
private:
    string name;
    int days;
    double rent;
    static const double rentPerDay;
public:
```

```

    Customer_Hotel_Mercato(string name, int days){
        this->name = name;
        this->days = days;
        rent = 0.0;
    }
    void calculateRent(){
        if (days > 7){
            rent = rentPerDay * (days-1);
        }
        else{
            rent = rentPerDay * days;
        }
    }
    void displayDetails() const{
        cout << "Customer Name: " << name << endl;
        cout << "Total Days: " << days << endl;
        cout << "Total Rent: " << rent << endl;
    }
};

const double Customer_Hotel_Mercato::rentPerDay = 1000.85;

int main(){
    Customer_Hotel_Mercato customer1("Aisha", 2);
    Customer_Hotel_Mercato customer2("Ali", 10);
    Customer_Hotel_Mercato customer3("Ahmed", 14);

    customer1.calculateRent();
    customer2.calculateRent();
    customer3.calculateRent();

    customer1.displayDetails();
    customer2.displayDetails();
    customer3.displayDetails();

    return 0;
}

```

OUTPUT:

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS C:\Users\PMLS\Desktop\NED\8th Semester\OOP\Lab4> g++ ex5.cpp -o ex5
```

```
PS C:\Users\PMLS\Desktop\NED\8th Semester\OOP\Lab4> ./ex5
```

```
Customer Name: Aisha
```

```
Total Days: 2
```

```
Total Rent: 2001.7
```

```
Customer Name: Ali
```

```
Total Days: 10
```

```
Total Rent: 9007.65
```

```
Customer Name: Ahmed
```

```
Total Days: 14
```

```
Total Rent: 13011.1
```

```
PS C:\Users\PMLS\Desktop\NED\8th Semester\OOP\Lab4> 
```